

LECTURE 5

BASIC OF
JAVASCRIPT

[HTTPS://WWW.MENTI.COM/](https://www.menti.com/)

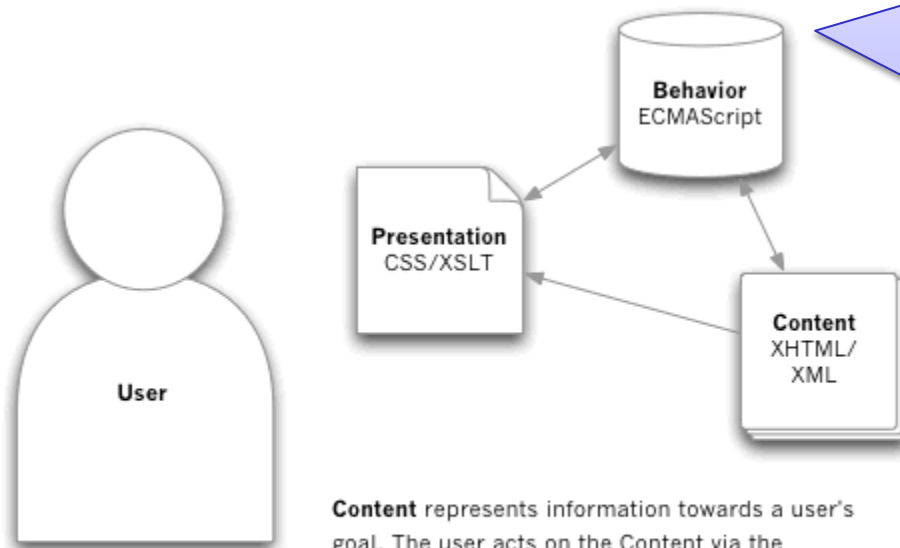
Why we need to Learn JavaScript?



JAVASCRIPT ORIGIN

ECMA Script are like rules and guideline

JavaScript is an implementation of the ECMAScript language standard and is typically used to enable programmatic access to computational objects within a **host environment**



Content represents information towards a user's goal. The user acts on the Content via the **Presentation** (the interface). The user's actions may trigger response actions, **Behaviors**, that may act on Content and/or Presentation.

Examples of ECMAScript:
JavaScript by Netscape (LiveScript),
Jscript by Microsoft, and
Action Script by Adobe.

[HTTPS://WWW.MENTI.COM/](https://www.menti.com/)

Enumerate the differences
between Java and JavaScript?



OVERVIEW OF JAVASCRIPT

- JavaScript is a scripting language to build more functional and interactive websites.
- JavaScript can be embedded in many different things, but its primary use is embedded in HTML documents
- **JavaScript** is a subset of Java
- JavaScript and Java are only related through syntax
 - JavaScript is dynamically typed
 - JavaScript's support for objects is very different
- Differences between Java and JavaScript:
 - Java is a **compiled language**
 - JavaScript is an **interpreted language**



OVERVIEW OF JAVASCRIPT

- The JavaScript Programming Language
 - JavaScript
 - Two formats:
 - **Client-side**
 - » Program runs on client (browser)
 - **Server-side**
 - » Program runs on server
 - » Proprietary to web server platform

JAVASCRIPT FRAMEWORK

- Angular
- React
- jQuery
- Vue.js
- Ext.js
- Ember.js

- Meteor
- Mithril
- Node.js
- Polymer
- Aurelia
- Backbone.js

WHAT JAVASCRIPT CAN DO?

- 1) JavaScript gives HTML designers **a programming tool**
- 2) JavaScript can put dynamic text into an HTML page
 - 1) A JavaScript statement like this: `document.write("<h1>" + name + "</h1>")` can **write a variable text into an HTML** page
- 3) JavaScript can react to events
 - 1) A JavaScript can be **set to execute when something happens**, like when a user clicks on an HTML element
- 4) JavaScript can read and write HTML elements
 - 1) A JavaScript can **read and change the content of an HTML element**
- 5) JavaScript can be used to validate data
 - 1) A JavaScript can be used to **validate form data** before it is submitted to a server-save the server from extra processing

WHAT JAVASCRIPT CAN DO?

- **Client side validation**
- **Manipulating HTML Pages**
- **User Notifications**
- **Back-end Data Loading** - JavaScript provides Ajax library which helps in loading back-end data while you are doing some other processing.
- **Presentations** - JavaScript also provides the facility of creating presentations which gives website look and feel.
- **Server Applications** - Node JS is built on Chrome's JavaScript runtime for building fast and scalable network applications.

Example JavaScript Change HTML Elements

1) JavaScript can change HTML content

- `document.getElementById("demo").innerHTML = " Hello JavaScript. ";`
- This example uses the method to "find" an HTML element (with `id="demo"`), and changes the element content (**innerHTML**) to "Hello JavaScript".

2) JavaScript can change HTML attribute

- `document.getElementById('myImage'); if (image.src.match("bulbon")) { image.src = "pic_bulboff.gif"; } else { image.src = "pic_bulbon.gif" } }`
- This example changes an HTML image, by changing the src attribute of an `<img>` tag. [Light bulb example.](#)

3) JavaScript can change HTML styles (CSS)

- `document.getElementById("demo").style.fontSize = "25px";`

JAVASCRIPT SYNTAX

- The HTML `<script>` tag is used to insert a JavaScript into an HTML page.

The `<script>` tags tell the browser to expect a script in between them.

```
<html>
<body>
<script type="text/javascript">
document.write("JavaScript is not Java");
</script>
</body>
</html>
```

This part that writes/displays the actual text

Where to put your scripts?

1. Scripts in `<head>`
2. Scripts in `<body>`
3. In both the body and the head section
4. Using an External JavaScript

Where to put your scripts? **<head>**.

```
<!DOCTYPE html>
<html><head>
<script>
function myFunction() {
    document.getElementById("demo").innerHTML = "Paragraph changed.";
}
</script>
</head>
<body>
<h1>My Web Page</h1>
<p id="demo">A Paragraph</p>
<button type="button" onclick="myFunction()">Try it</button>
</body>
</html>
```

Where to put your scripts? **<body>**.

```
<!DOCTYPE html>
<html>
  <body>
    <h1>My Web Page</h1>
    <p id="demo">A Paragraph</p>
    <button type="button" onclick="myFunction()">Try it</button>
    <script>
      function myFunction() {
        document.getElementById("demo").innerHTML = "Paragraph changed.";
      }
    </script>
  </body>
</html>
```

Where to put your scripts? **<head>** and **<body>**

```
<html>
  <head>
    <script>
      function message()
      {
        alert("This alert box was called with the onload event");
      }
    </script>
  </head>
  <body onload="message()">
    <script>
      document.write("This message is written by JavaScript");
    </script>
  </body>
</html>
```

Where to put your scripts? **External javascript**

```
<html>
```

```
  <head>
```

```
    <script src="myscript.js">
```

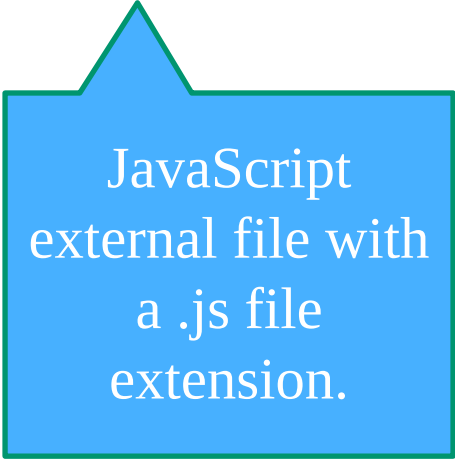
```
  </script>
```

```
  </head>
```

```
  <body>
```

```
  </body>
```

```
</html>
```



JavaScript
external file with
a .js file
extension.

Placing JavaScripts in external files has some advantages:

- It separates HTML and code
- It makes HTML and JavaScript easier to read and maintain
- Cached JavaScript files can speed up page loads

Writing Output to the Web Page

To write text to a Web page, use the following commands:

- Writing into an alert box, using **window.alert("text")**.

[Click here for example.](#)

- Writing into the HTML output using **document.write(5+6)**.

No new line added
after each statement

[Click here for example.](#)

- Writing into the HTML output using **document.writeln(5+6)**.

Add new line after
each statement

[Click here for example.](#)

- Writing into an HTML element, using **innerHTML**.
- Writing into the browser console, using **console.log("text")**. [Click here for example.](#)

JAVASCRIPT STATEMENTS

- JavaScript statements are composed of:
 - Values (variables), Operators, Expressions, Keywords, and Comments.
- JavaScript statements are separated by **semicolons (;)**

JAVASCRIPT VARIABLES

Rules for variable names:

- Variable names are **case sensitive**
- They **must begin with a letter or underscore character**
- You can create a variable **with the var statement**:
var strname = some value;
- You can also create a variable **without the var statement**:
strname = some value;
- Assign a Value to a Variable
var strname = "Hege";
Or like this:
strname = "Hege";

```
<html>
<body>
<script type="text/javascript">
```

```
var firstname;
```

Declare a variable “firstname”

```
firstname="Toot";
```

Assign a value to “firstname”

```
document.write(firstname);
```

```
document.write("<br />");
```

```
firstname="Tweet";
```

```
document.write(firstname);
```

```
</script>
```

<p>The script above declares a variable, assigns a value to it, displays the value, changes the value, and displays the value again.</p>

```
</body>
```

```
</html>
```

Working with Variables and Data

- JavaScript **variable types**:
 - **Numeric** variables- 1,2,3,
 - **String** variables- "hello"
 - **Boolean** variables - true and false
 - **Null** variables- no value at all
- You must **declare** a variable before using it

Working with Expressions and Operators

- **Expressions:** An expression is a combination of values, variables, and operators, which computes to a value.
- **Operators** are elements that perform actions within expressions
 1. **Arithmetic Operators** (+,-,*)
 2. **Assignment Operators** (=,+=)
 3. **Comparison Operators** (==,>,<)
 4. **Logical/boolean Operators** (&&,!)
 5. **String Operators**

Examples

Example 1(using "+"):

```
txt1 = "John";  
txt2 = "Doe";  
txt3 = txt1 + " " + txt2;
```



The result of **txt3** will be:

```
John Doe
```

Example 2(using "+=") :

```
txt1 = "What a very ";  
txt1 += "nice day";
```



The result of **txt1** will be:

```
What a very nice day
```

Example 3:

```
x = 5 + 5;  
y = "5" + 5;  
z = "Hello" + 5;
```



The result of **x, y, and z** will be:

```
10  
55  
Hello5
```

JAVASCRIPT COMMENTS

- Not all JavaScript statements are "executed".
- Code after double slashes `//` or between `/*` and `*/` is treated as a **comment**.
- Comments are ignored, and will not be executed:

Example:

```
var x = 5;    // I will be executed
```

```
// var x = 6;    I will NOT be executed
```

JAVASCRIPT FUNCTIONS

- A function is a group of reusable code which can be called anywhere in your program.
- This eliminates the need of writing the same code again and again.
- Functions allow a programmer to divide a big program into a number of small and manageable functions.

Creating JavaScript Functions

- A **function** is a series of commands that perform an action or calculates a value
- A **function name** identifies a function
- **Parameters** are values used by the function

Syntax

```
function functionname(var1,var2,...,varX)  
{  
  code to be executed  
}
```

Example JavaScript Function

```
<!DOCTYPE html>
<html>
<body>

<p>This example calls a function which
performs a calculation, and returns the
result:</p>

<p id="demo"></p>

<script>
function myFunction(a, b) {
    return a * b;
}
document.getElementById("demo").innerHTML
= myFunction(4, 3);
</script>

</body>
</html>
```

Result:

This example calls a function which
performs a calculation, and returns the
result:

12

JavaScript Function Invocation

- The code inside the function will execute when "something" **invokes** (calls) the function:
 - When an event occurs (when a user clicks a button)
 - When it is invoked (called) from JavaScript code
 - Automatically (self invoked)
- Why use functions?
 - You can reuse code: Define the code once, and use it many times.

More Function Example:

```
<!DOCTYPE html>
<html>
<body>
<p id="demo"></p>

<script>
function toCelsius(fahrenheit) {
    return (5/9) * (fahrenheit-32);
}
var text = "The temperature is " +
toCelsius(77) + " Celsius";
document.write(text);

document.getElementById("demo").innerHTML =
"The temperature is " + toCelsius(77) + "
Celsius";
</script>
</body>
</html>
```

Result:

The temperature is 25 Celsius

The temperature is 25 Celsius

FUNCTION

- A function in JavaScript can be either **named** or **anonymous**.
- How to define a anonymous function?
 - An **anonymous function** can be defined in similar way as a normal function but it would not have any name.
 - Functions stored in variables do not need function names. They are always invoked (called) using the variable name.

```
var x = function (a, b) {return a * b};  
var z = x(4, 3);
```

[HTTPS://WWW.MENTI.COM/](https://www.menti.com/)

what is object in javaScript?



JAVASCRIPT OBJECTS

In JavaScript, almost "everything" is an object.

- Booleans can be objects (if defined with the new keyword)
- Numbers can be objects (if defined with the new keyword)
- Strings can be objects (if defined with the new keyword)
- Dates are always objects
- Maths are always objects
- Regular expressions are always objects
- Arrays are always objects
- Functions are always objects
- Objects are always objects

JAVASCRIPT OBJECTS

- Objects are variables too. But objects can contain many values.
- This code assigns **many values** (Fiat, 500, 850kg, white) to a **variable** named car:

Object	Properties	Methods
	car.name =	car.start()
	Fiat	car.drive()
	car.model =	car.brake()
	500	car.stop()
	car.weight =	
	850kg	
	car.color =	
	white	

JS OBJECTS CONT.

- JavaScript objects **syntax**:

```
var car = {name:"Fiat", model:500, weight:"850kg",  
color:"white"};
```

- Object Properties**

- The name:values pairs (in JavaScript objects) are called **properties**.

Property	Property value
Name	Fiat
Model	500
Weight	850kg
color	white

JS OBJECTS CONT.

- **Accessing Object Properties**
 - Object properties can be access in two ways:

`objectName.propertyName` OR `objectName["propertyName"]`

```
<script>
var person = { firstName: "John",
lastName: "Doe", id: 5566 };

document.getElementById("demo").innerHTML
= person.firstName + " "+person.lastName;

document.write (person["firstName"] + " "
+ person["lastName"]);
</script>
```

Output:

John Doe

John Doe

[HTTPS://WWW.MENTI.COM/](https://www.menti.com/)

Give Example of JavaScript
Events.



JAVASCRIPT EVENT HANDLER

- JavaScript are often event-driven. That is, a piece of codes (called event handler) fires/ triggers in response to a certain user's or browser's action, which generates an event.
- The commonly-used events are:
 1. **click**: generated when the user clicks on an HTML element.
 2. **mouseover, mouseout**: generated when the user positions the mouse pointer inside/away from the HTML element.
 3. **load, unload**: generated after the browser loaded a document, and before the next document is loaded, respectively.

What can JavaScript events do?

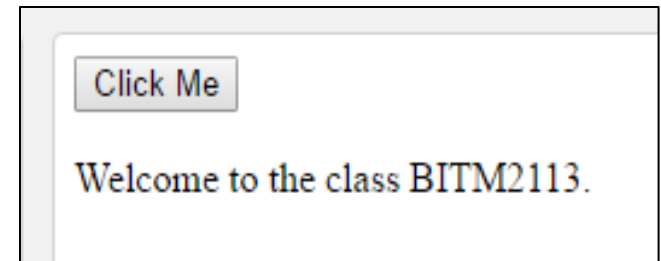
- Event handlers can be used to handle, and verify, user input, user actions, and browser actions:
 - Things that should be done every time a page loads
 - Things that should be done when the page is closed
 - Action that should be performed when a user clicks a button
 - Content that should be verified when a user inputs data
- Many different methods can be used to let JavaScript work with events:
 - HTML event attributes can execute JavaScript code directly
 - HTML event attributes can call JavaScript functions
 - You can assign your own event handler functions to HTML elements
 - You can prevent events from being sent or being handled

Example:

When the button is clicked, trigger myFunction() with an event.

```
<!DOCTYPE html>
<html>
<body>
<button onclick="myFunction()">Click
Me</button>
<p id="demo"></p>
<script>
function myFunction() {
document.getElementById("demo").innerHTML
= "Welcome to the class BITM2113."; }
</script>
</body>
</html>
```

Output:



JAVASCRIPT DATES

- The Date object lets you work with dates (years, months, days, hours, minutes, seconds, and milliseconds)
- A JavaScript date can be written as a string:
 - **Fri Mar 18 2016 13:08:54 GMT+0800 (Malay Peninsula Standard Time)**
- or as a number:
 - **1458277734226**
 - Dates written as numbers, specifies the number of milliseconds since January 1, 1970, 00:00:00.

Syntax

```
<script>
document.getElementById
("demo").innerHTML =
Date();
</script>
```

```
<script>
var d = new Date();
document.getElementById
("demo").innerHTML = d;
</script>
```

Result:

Fri Mar 18 2016 13:18:19 GMT+0800
(Malay Peninsula Standard Time)

JAVASCRIPT ARRAY

- An **array** is an ordered collection of values referenced by a single variable name which can hold more than one value at a time.

`var array-name = [item1, item2,...];`

- **Example:**

```
var points = new Array();    // Bad
var points = [];            // Good
```

```
var points = new Array(40, 100, 1, 5, 25, 10) // Bad
var points = [40, 100, 1, 5, 25, 10];         // Good
```

Tips:

For simplicity, readability and execution speed, use the second way.

- JavaScript array can store different kind of objects.

```
var faq = new Array(3);
faq[0] = "What are JavaScript arrays";
faq[1] = Date.now;
faq[2] = 5000;
```

JAVASCRIPT ARRAY CONT.

- Adding Array Elements:

```
var fruits = ["Banana", "Orange", "Apple", "Mango"];  
fruits.push("Lemon");           // adds a new element  
(Lemon) to fruits
```

```
var fruits = ["Banana", "Orange", "Apple", "Mango"];  
fruits[fruits.length] = "Lemon"; // adds a new  
element (Lemon) to fruits
```

```
var fruits = ["Banana", "Orange", "Apple", "Mango"];  
fruits[10] = "Lemon";           // adds a new  
element (Lemon) to fruits
```

CONDITIONAL STATEMENTS

- **Conditional statements** are commands that run only when specific conditions are met
- Conditional statements require a **Boolean expression**
 - you need one of the following operators to create a Boolean (true/false) expression:
 - Comparison operator
 - Logical operator
 - Conditional operator

Comparison Operators and Logical Operators

Comparison Operators

Operator	Description
<code>==</code>	Equal to
<code>===</code>	equal value and equal type
<code>!=</code>	not equal
<code>!==</code>	not equal value or not equal type
<code>></code>	greater than
<code><</code>	less than
<code>>=</code>	greater than or equal to
<code><=</code>	less than or equal to

Logical Operators

Operator	Description
<code>&&</code>	and
<code> </code>	or
<code>!</code>	not

CONDITIONAL STATEMENTS

- Conditional statements in JavaScript are used to perform different actions based on different conditions.
- In JavaScript we have four conditional statements:

1) if statement - use this statement if you want to execute a set of code when a condition is true

```
if (condition) {  
    block of code to be executed if the  
condition is true  
}
```

Eg:

```
if (hour < 18) {  
    greeting = "Good day";  
}
```

2) if...else statement - use this statement if you want to select one of two sets of lines to execute

```
if (condition) {  
    block of code to be executed if the  
condition is true  
} else {  
    block of code to be executed if the  
condition is false  
}  
if (hour < 18) {  
    greeting = "Good day";  
} else {  
    greeting = "Good evening";  
}
```

CONDITIONAL STATEMENTS cont.

3) switch statement - use this statement if you want to select one of many sets of lines to execute

Syntax:

```
switch(expression) {  
    case n:  
        code block  
        break;  
    case n:  
        code block  
        break;  
    default:  
        default code block  
}
```

Example:

```
switch (new Date().getDay()) {  
    case 6:  
        text = "Today is Saturday";  
        break;  
    case 0:  
        text = "Today is Sunday";  
        break;  
    default:  
        text = "Looking forward to the  
Weekend";  
}
```

4) If and If...else Statement

You should use the if statement if you want to execute some code if a condition is true.

JAVASCRIPTS LOOPING

- In JavaScript we have the following looping statements:

1) The For Loop - run statements a specified number of times

Syntax:

```
for (statement 1; statement 2; statement 3)  
{  
    code block to be executed  
}
```

Example:

```
for (i = 0; i < 5; i++) {  
    text += "The number is " + i + "<br>";  
}
```

2) The While Loop - loops through a block of code while a condition is true

Syntax:

```
while (condition) {  
    code block to be executed  
}
```

Example:

```
while (i < 10) {  
    text += "The number is " + i;  
    i++;  
}
```

3) The do...while loop - loops through a block of code once, and then repeats the loop while a condition is true

Syntax:

```
do {  
    code block to be executed  
}  
while (condition);
```

Example:

```
do {  
    text += "The number is " + i;  
    i++;  
}  
while (i < 10);
```

ERRORS IN JAVASCRIPT

There are three types of errors:

1

Load time errors: Errors which come up when loading a web page like improper syntax errors are known as Load time errors and it generates the errors dynamically.

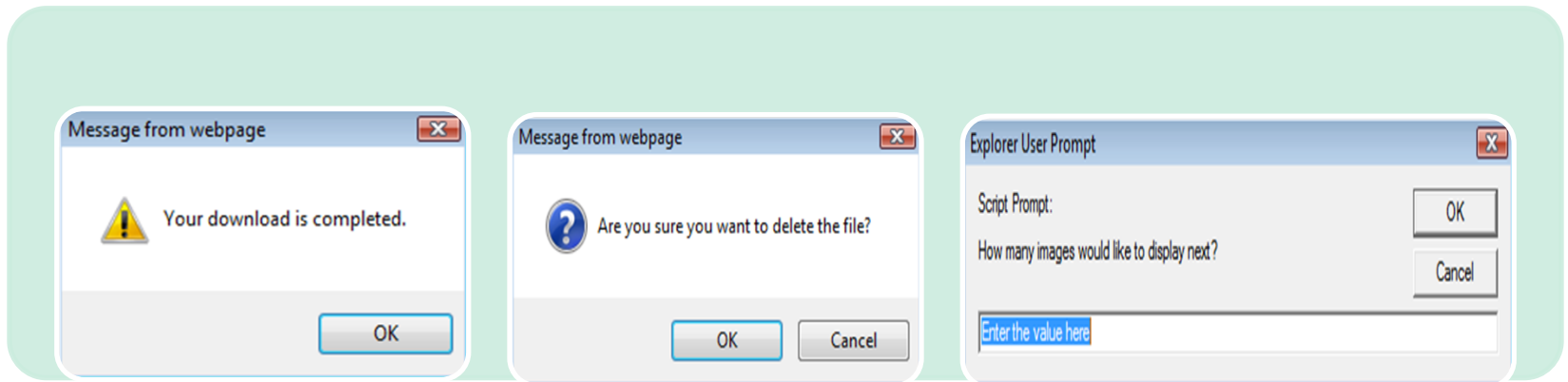
2

Run time errors: Errors that come due to misuse of the command inside the HTML language.

3

Logical Errors: These are the errors that occur due to the bad logic performed on a function which is having different operation.

JAVASCRIPT POPUP BOXES

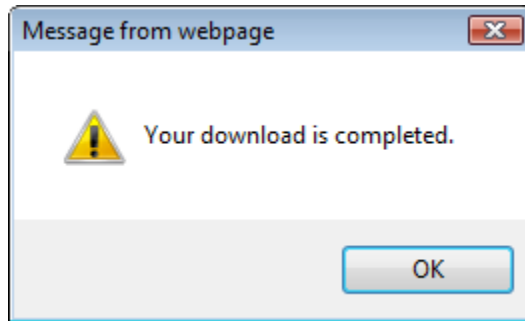


Alert Box
`alert("Message");`

Confirm Box
`confirm("Message");`

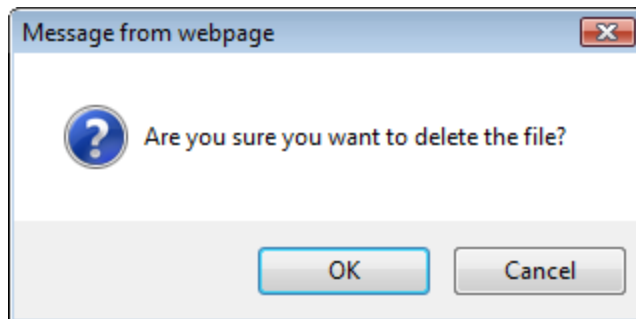
Prompt Box
`prompt("Message", "Default
response");`

JAVASCRIPT POPUP BOXES



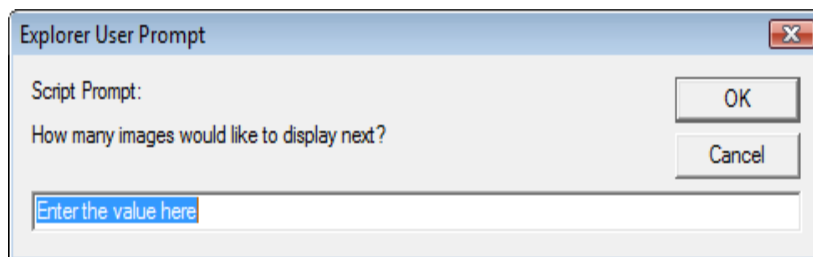
Alert Box

- used if you want to make sure information comes through to the user.
- `alert("Message");`



Confirm Box

- used if you want the user to verify or accept something
- `confirm("Message");`



Prompt Box

- used if you want the user to input a value before entering a page
- `prompt("Message", "default response");`

[HTTPS://WWW.MENTI.COM/](https://www.menti.com/)

**Which words would you use
to describe today's lecture?**



QUESTION / COMMENT?

